

Java Full Stack – Agile Methodology Scenario Based Questions & Answers

Agile & Sprint Planning (Q1–Q10)

Q1. During sprint planning, the PO asks you to deliver a new feature, but backend API contracts are not finalized. How do you proceed?

Negotiate scope → build frontend with mock API responses using Postman/MockServer → integrate with real API once backend is ready. Agile encourages incremental progress.

Q2. In daily stand-up, your teammate says their backend task is blocked due to unclear DB schema. What should you do?

Offer to pair-program/debug, escalate to Scrum Master if dependency on another team, and suggest creating DB schema migration scripts with Liquibase/Flyway.

Q3. You are asked to estimate a user story where both frontend (React) and backend (Spring Boot) changes are needed. How do you give estimation?

Break into subtasks (UI, API, DB, integration, testing) → give story points for combined effort → add buffer for integration testing.

Q4. Mid-sprint, requirements change slightly. The PO says add one extra field in response. What will you do?

Assess effort → if < 1 day, accommodate within sprint. If bigger, log a Change Request for next sprint. Agile = flexible but also controlled.

Q5. What do you do if you finish your sprint tasks early?

Pick up bugs from backlog, help teammates, or refactor/test existing code. Agile values team contribution > individual work.

Q6. A QA raises a bug that's not in acceptance criteria. Do you fix it in current sprint?

If critical/blocker → fix immediately. If cosmetic/minor → add to backlog → prioritize next sprint.

Q7. How do you handle dependencies between frontend & backend teams in Agile?

Use API contracts (Swagger/OpenAPI), mock responses, parallel development. Align in sprint planning to avoid blockers.

Q8. PO asks why velocity dropped in last sprint. What would you explain?

Reasons: production hotfix, unplanned dependencies, onboarding new team members. Agile encourages transparency in retrospective.

Q9. During sprint demo, your feature works locally but fails in staging. How do you explain?

Acknowledge → highlight environment differences (config, DB, CORS, API URLs) → plan quick fix. Agile = working software demo is key.

Q10. You are assigned a spike story (research task). How do you deliver it?

Explore → create POC/notes → present findings in retro/demo → create new stories for implementation.

Backend (Spring Boot, APIs, DB) (Q11–Q20)

Q11. A story requires fetching student details from DB and exposing via REST API. Which Spring Boot components do you create?

Entity → Repository (JPA) → Service layer (business logic) → Controller (REST API) → Integration test.

Q12. The team is migrating from monolith to microservices. How do you split services in Agile?

Break by business capability (e.g., User Service, Payment Service) → define APIs → migrate gradually. Each sprint delivers one service.

Q13. A bug is reported: API returns 500 error in production. What's your debugging approach?

Check logs → reproduce locally with same input → debug DB queries → fix & write test → hotfix deployment.

Q14. Story: add caching to API for performance. How do you implement?

Use Spring Cache + Redis → configure TTL → write tests → validate in load testing. Agile = optimize incrementally.

Q15. DB migration is needed. How do you handle in Agile?

Create migration script (Liquibase/Flyway) → review in sprint → run in staging → deploy with CI/CD pipeline.

Q16. API performance degrades under load. What actions do you take?

Profile code → add DB indexing → caching → async calls → load testing → optimize. Raise performance improvements as technical debt stories.

Q17. During integration, frontend expects JSON field fullName but backend sends name. How do you handle?

Fix API contract → if breaking change, use DTO mapping (ModelMapper) → version API if multiple clients impacted.

Q18. Security vulnerability reported (SQL injection). How do you solve?

Use prepared statements/JPA parameter binding → sanitize inputs → write security tests → re-deploy. Agile ensures quick fix + learning.

Q19. How do you handle logging in a real-time Agile project?

Use SLF4J + Logback/Log4j → structured logs → centralized (ELK, Splunk) → log correlation ID for tracing.

Q20. Team asks you to write unit tests for backend API. What do you use?

JUnit + Mockito + Spring Boot Test → follow TDD/BDD in Agile → ensure >80% coverage.

Frontend (React/Angular + Integration) (Q21–Q30)

Q21. Story: Create student dashboard with API data. How do you integrate?

Use fetch/axios in React → call /students API → store in state (Redux/useState) → display UI → error handling.

Q22. Backend API takes 5s. PO asks for better UX. What do you do?

Show loading spinner, use async calls, and discuss with backend team about caching/pagination. Agile values user feedback.

Q23. Requirement: Add role-based UI access. How do you design?

Fetch user role from JWT → conditionally render components → secure routes → sync with backend authorization.

Q24. Mid-sprint, designer changes button style. How do you react?

If minor (CSS only) → fix in sprint. If large refactor → log new story for next sprint.

Q25. A bug: form not submitting. How do you debug?

Check browser console → network API calls → React state handling → fix root cause. Agile encourages fast bug fixing.

Q26. Sprint review shows UI not mobile-friendly. How do you fix?

Add responsive CSS/Bootstrap/Material UI → test on devices → demo in next sprint.

Q27. How do you manage environment configs (dev, QA, prod) in frontend?

Use .env files → inject via build pipeline → don't hardcode URLs.

Q28. A requirement is to display real-time updates. How do you handle?

Use WebSockets/Server-Sent Events → React state → update UI live.

Q29. How do you write frontend tests in Agile?

Use Jest + React Testing Library → write snapshot/unit tests → CI integration.

Q30. How do you ensure accessibility (a11y) in Agile projects?

Add ARIA roles → keyboard navigation → screen reader support → include accessibility testing in DoD.

DevOps, CI/CD, Agile Collaboration (Q31–Q40)

Q31. How do you deploy a Spring Boot app in Agile projects?

Use Docker → CI/CD (Jenkins/GitHub Actions) → deploy to AWS/GCP/Azure → automate rollback.

Q32. What do you do if deployment fails mid-sprint?

Rollback → hotfix → root cause analysis → retrospective discussion → improve pipeline.

Q33. How do you manage secrets (DB passwords, API keys)?

Store in Vault/KMS/Environment variables → never commit to Git.

Q34. Agile retrospective reveals too many hotfixes. What actions do you take?

Improve testing coverage, add QA automation, add Definition of Done stricter checks.

Q35. You need to monitor app health after release. What do you do?

Use Actuator in Spring Boot → expose health endpoints → integrate with Prometheus + Grafana.

Q36. Sprint backlog shows many bugs. What do you suggest?

Add bug-fixing sprint or allocate fixed capacity (20–30%) each sprint for bugs.

Q37. During daily scrum, a teammate is always silent. As Agile dev, what do you do?

Encourage sharing blockers → ask open questions → escalate to Scrum Master if persistent.

Q38. Stakeholders ask for mid-sprint demo. What's your response?

Agree for partial demo → show working increments → explain unfinished items transparently.

Q39. How do you track progress in Agile full stack project?

Use JIRA board → track stories, burndown chart → daily stand-ups.

Q40. PO asks why Agile is better than Waterfall for Java Full Stack projects. What do you explain?

Agile → continuous feedback, parallel dev (UI + API mocks), frequent releases, fast bug fixes. Waterfall → rigid & late feedback.